PROJECT REPORT

EDU-TRACK

STUDENT ACADEMIC MANAGEMENT SYSTEM

A Full-Stack Academic Management Web Application

Submitted by

SYSTEM SAVIOURS

Academic Project Report | 2026

S. No.	Team member	Register number
1	K. Srikanth	99230040602
2	V. Pavan Kumar	99240040511
3	D.V. Nithin Reddy	99230040526
4	M. Bala Maruthi	99240040608
5	K. Hemasai Latha	99230040122
6	J. Charitha	99230040112

ABSTRACT

Edu-Track is a web-based Student Academic Management System designed to bring important academic activities of an educational institution into a single digital platform. The system provides separate role-based workspaces for administrators, faculty members and students. It supports the management of departments, sections, subjects, student and faculty profiles, teaching assignments, class timetables, attendance, assignments, submissions, internal assessments, marks, examination schedules, online examinations, study materials and announcements.

The application is implemented using React and TypeScript for the frontend, with a Node.js and Express backend. The source code defines MongoDB data models through Mongoose and includes a persistent JSON-file storage layer that remains available when a MongoDB connection is not active. JWT-based sessions are used for application authentication, while Firebase Authentication with Google sign-in is integrated for the administrator sign-in flow. Socket.IO supports real-time event communication between the server and connected clients.

Edu-Track aims to reduce manual academic record handling, improve coordination among institutional roles, provide timely access to academic information and support data-informed monitoring of attendance and student performance. Its assessment analytics include the identification of concepts where student performance may require additional academic support.

This report describes the project objectives, requirements, architecture, workflows, functional modules, data design, implementation approach, security considerations, test scenarios, advantages, limitations and possible future enhancements. Descriptions are based on the supplied project presentation and source-code archive; where a feature is defined in the code but depends on deployment configuration or external services, that dependency is stated.

TABLE OF CONTENTS

S.NO	Content	Page.No
1	Introduction	4
2	Problem Statement	4
3	Objectives and Scope	5
4	Existing and Proposed Systems	6
5	System Requirements and Feasibility	6
6	System Architecture	8
7	Methodology and System Workflow	9
8	Functional Modules	10
9	Database Design	12
10	Technology Stack	13
11	Implementation Details	14
12	Authentication, Security and Privacy	16
13	Testing Strategy and Test Cases	17
14	Advantages and Applications	18
15	Limitations and Future Enhancements	19
16	Conclusion	19
17	References	20

1. INTRODUCTION

Educational institutions manage a large amount of academic information every semester. This includes student and faculty profiles, department and section details, course registrations, teaching assignments, timetables, attendance records, assignments, internal marks, examination schedules and learning resources. When these activities are handled through separate spreadsheets, paper registers or disconnected applications, maintaining accurate and updated information becomes difficult.

Edu-Track addresses this need through a centralized academic management application. It brings administrators, faculty members and students into a shared system while presenting each role with functions relevant to its responsibilities. Administrators manage the institutional structure and monitor academic activity. Faculty members manage the academic work assigned to them, such as attendance, assignments, marks, study materials and examinations. Students use the portal to view their academic information, schedules, attendance, learning resources, assessments and results.

The supplied presentation identifies the project as a MERN-based academic portal. Inspection of the supplied source code shows a React and TypeScript client, an Express and Node.js backend, Mongoose model definitions for MongoDB, and a JSON-backed persistent storage mechanism with MongoDB synchronization when a database connection is available. This report documents that implemented arrangement rather than describing the project as a MongoDB-only system.

1.1 Purpose of the System

- Centralize routine academic information and operations in one web application.
- Provide role-specific access for Admin, Faculty and Student users.
- Reduce repeated manual entry and make academic records easier to retrieve.
- Support academic monitoring through attendance, marks, results and analytics.
- Improve communication through announcements, study materials and real-time updates.

1.2 Intended Users

User role	Main responsibilities
Administrator	Manage departments, sections, subjects, user accounts, faculty/student records, assignments, timetables, examinations and institutional monitoring.
Faculty	Access assigned classes and subjects; record attendance; create assignments and assessments; evaluate submissions; publish marks; upload materials and announcements.
Student	View profile, enrolled subjects, timetable, attendance, assignments, study materials, announcements, examination information, marks and results.

2. PROBLEM STATEMENT

Academic administration involves multiple connected processes, but these processes are often maintained separately. Manual attendance registers, spreadsheets for marks, printed timetables and informal communication channels can result in duplicated work, delayed updates and difficulty obtaining a consolidated view of student progress.

2.1 Identified Problems

- Fragmented academic records: student, faculty, subject and attendance information may be maintained in separate places.
- Time-consuming administration: routine updates to sections, subjects, schedules and user records require repeated manual effort.

- Limited visibility: students may not have a single place to check attendance, assignments, results and announcements.
- Delayed feedback: assignment evaluation and marks publication may not be visible to students immediately.
- Difficult performance monitoring: identifying attendance shortages or weak academic concepts requires collecting and comparing information manually.
- Communication gaps: academic materials, timetable changes and notices may not reach the intended group in a consistent way.

2.2 Problem Definition

A centralized, role-based academic management system is required to organize institutional data and support the academic workflows of administrators, faculty members and students. The system should provide consistent access to records, simplify routine operations and make relevant academic information available through a common interface.

3. OBJECTIVES AND SCOPE

3.1 Objectives

- Develop a centralized web application for academic administration.
- Provide separate login and portal experiences for Admin, Faculty and Student roles.
- Manage departments, sections, subjects, student profiles, faculty profiles and teaching assignments.
- Create and display class timetables and examination schedules.
- Record attendance and calculate attendance percentages.
- Create assignments, accept student submissions and support faculty evaluation and feedback.
- Manage internal assessments, marks, grades, SGPA and CGPA-related result information.
- Conduct scheduled online examinations with timed attempts and result evaluation.
- Provide access to study materials, announcements and notifications.
- Generate institutional and academic analytics, including attendance shortage and weak-concept indicators.
- Support real-time updates for relevant academic events through Socket.IO.

3.2 Scope

The project scope covers academic administration from the setup of institutional entities through day-to-day teaching and student-facing academic activities. It includes role-based portals, record management, attendance, assessments, online examinations, timetable management, study resources, announcements and analytics. The source code also defines audit-log, notification, examination-attempt and academic-result data structures.

The project is a web application. The supplied source code does not establish a dedicated native Android or iOS application. The online examination interface includes timed attempts and automatic evaluation logic for objective questions; the presentation explicitly states that camera or webcam monitoring is not included.

3.3 Scope Boundaries

- The system is intended for academic information and workflow management, not as a replacement for every institutional ERP or finance system.
- External authentication, cloud database connectivity and deployment depend on valid configuration and service availability.
- The source archive includes demo/seed data and local persistence logic; production use requires institution-specific configuration, security review and operational testing.

4. EXISTING AND PROPOSED SYSTEMS

4.1 Existing System

In a conventional academic workflow, departments, student details, faculty assignments, attendance, marks and schedules may be maintained using paper documents, spreadsheets or separate applications. Faculty may communicate assignments and materials through different channels, while students may need to contact staff or consult multiple sources to obtain updated information.

4.2 Proposed System

Edu-Track proposes a centralized web portal with role-specific dashboards. Institutional data is organized through connected academic entities such as departments, sections, subjects, profiles and teaching assignments. Faculty can manage attendance and academic activities for their assigned classes, while students can view their own academic information. The system also includes online examinations, announcements, study materials and analytics.

4.3 Comparison

Aspect	Conventional approach	Edu-Track approach
Academic records	Paper files and separate spreadsheets	Centralized application data and role-specific views
Attendance	Manual registers or independent sheets	Digital attendance sessions and percentage recalculation
Timetable	Printed or separately circulated schedules	Timetable module with department, semester and section context
Assignments	Manual distribution and collection	Assignment creation, online submission and evaluation workflow
Examinations	Separate notices and assessment processes	Exam schedules and timed online exam module
Communication	Multiple informal channels	Announcements, materials and real-time event notifications
Performance review	Manual calculation and comparison	Marks, result records and academic analytics

4.4 Expected Improvement

The proposed system is intended to reduce duplicated effort, improve access to current academic information, strengthen coordination among institutional roles and provide a more structured basis for monitoring student performance. These are design goals; actual time savings and performance improvements should be measured during institutional deployment.

5. SYSTEM REQUIREMENTS AND FEASIBILITY

5.1 Functional Requirements

- The system shall authenticate users and identify their role as Admin, Faculty or Student.
- Administrators shall be able to manage departments, sections, subjects, student and faculty information.
- The system shall support teaching assignments linking faculty, subjects, semester and section.
- Authorized users shall be able to view or manage class and examination timetables.
- Faculty shall be able to record attendance for a class session and its student records.
- Faculty shall be able to create assignments and assess student submissions.

- The system shall support internal assessment creation and mark entry.
- Faculty shall be able to create and publish online examinations with questions, duration and result settings.
- Students shall be able to attempt eligible published examinations and view released results.
- Faculty shall be able to upload study materials and publish announcements.
- Students shall be able to view relevant academic resources and notices.
- The system shall provide institutional analytics and audit-log information.

5.2 Non-Functional Requirements

Requirement	Description
Security	Role-aware access, token verification, password hashing and protected API operations.
Usability	Clear navigation and separate interfaces for administrators, faculty and students.
Performance	Efficient retrieval and update of academic records; real-time events for selected updates.
Reliability	Persistent local data file with optional MongoDB synchronization; deployment must be tested for recovery and consistency.
Maintainability	TypeScript types, separated UI components, route modules, models and service layers.
Scalability	MongoDB models and modular API routes provide a basis for extension; real capacity depends on deployment and database design.
Compatibility	Browser-based interface built with React and Vite.

5.3 Hardware and Software Requirements

Category	Minimum / recommended environment
Client device	Desktop or laptop; modern browser; internet or local network access.
Development runtime	Node.js runtime compatible with the installed dependencies; npm.
Frontend	React, TypeScript, Vite and a modern web browser.
Backend	Node.js, Express, TypeScript execution through tsx.
Database	MongoDB instance for database synchronization; JSON file storage is available in the application code.
Development tools	Visual Studio Code or another TypeScript-capable editor; terminal; Git (optional).

5.4 Feasibility Study

Technical Feasibility

The system uses established JavaScript and TypeScript technologies. React supports reusable user interfaces, Express provides HTTP API routing, Mongoose defines MongoDB schemas, and Socket.IO supports event communication. These technologies are suitable for a modular academic portal.

Operational Feasibility

The three role-based portals align with common institutional responsibilities. Admin, Faculty and Student users are presented with different functions, reducing the need for every user to navigate the entire application.

Economic Feasibility

The core development stack is based largely on open-source software. Actual operating costs may arise from hosting, MongoDB infrastructure, authentication services, domain registration, backups and maintenance.

Schedule Feasibility

The project is divided into modules that can be developed and tested incrementally: authentication, institutional setup, faculty/student portals, attendance, assessments, examinations, materials, announcements and analytics.

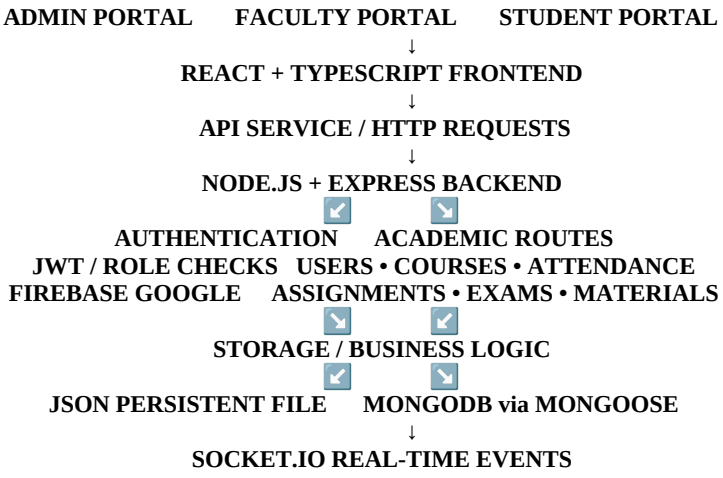
6. SYSTEM ARCHITECTURE

Edu-Track follows a layered web client-server design. The React frontend presents role-specific pages and communicates with the Express API. The backend applies authentication and role checks, executes academic business logic, accesses the persistent storage layer and emits selected real-time events through Socket.IO.

6.1 Architecture Layers

Layer	Technology / component	Responsibility
Presentation	React 19, TypeScript, Vite, CSS, Lucide React, Recharts	Renders landing, login, Admin, Faculty and Student interfaces; manages navigation and UI state.
Client service	EduTrackApi, API service, Socket service	Calls backend endpoints, maintains client-side cached state and subscribes to real-time events.
API / application	Node.js, Express, TypeScript	Exposes REST endpoints for authentication, academic records, attendance, assessments, materials and analytics.
Authentication	JWT, bcryptjs, Firebase Authentication / Firebase Admin	Supports application sessions, password hashing and Google sign-in for the administrator flow.
Business logic	Route handlers and storage service	Validates requests, updates records, recalculates attendance and results, and creates audit entries.
Persistence	JSON file store and Mongoose models / MongoDB	Maintains local persistent data and synchronizes records to MongoDB when a connection is available.
Real-time communication	Socket.IO and socket.io-client	Broadcasts selected academic events to connected clients and relevant user groups.

6.2 High-Level Architecture Diagram



6.3 Architecture Flow

1. The user opens Edu-Track in a web browser and selects the relevant login flow.
2. The frontend authenticates the user and determines the role-specific portal to display.
3. User actions are passed through the frontend API service to the Express backend.
4. Authentication middleware identifies the request user; protected routes apply the required role checks.
5. The route handler validates the request and performs the requested academic operation.
6. The storage service updates the persistent JSON data and, when MongoDB is connected, initiates synchronization with Mongoose models.
7. An API response is returned to the frontend. For supported events, Socket.IO sends updates to connected clients.

6.4 Storage Architecture Note

The supplied source code does not implement a MongoDB-only persistence path. The storage service reads and writes a JSON data file named edutrack_db.json and initiates background synchronization to MongoDB when the Mongoose connection is active. The health endpoint reports whether MongoDB is connected or the persistent embedded store is active. Therefore, the report describes this as a dual-mode persistence design. Deployment and testing should verify which mode is active and whether synchronization is complete for the required records.

7. METHODOLOGY AND SYSTEM WORKFLOW

7.1 Development Methodology

A modular and iterative development approach is appropriate for Edu-Track. Each major academic function can be implemented as a separate frontend component group and backend route module. Shared types define the structure of academic records, while the API service coordinates client-side operations. Testing can be performed module by module and then through end-to-end role-based workflows.

1. Requirement identification and role analysis
2. UI planning and component design
3. Data model and API design
4. Implementation of authentication and core academic modules
5. Integration of frontend, backend, storage and real-time events
6. Functional, integration and security testing
7. Deployment configuration and maintenance

7.2 Main User Workflow

Stage	Admin	Faculty	Student
Access	Signs in to administrative portal.	Signs in to faculty portal.	Signs in to student portal.
Academic setup	Maintains departments, sections, subjects and accounts.	Views assigned subjects and sections.	Views own profile, semester and subjects.
Teaching operations	Oversees schedules and institutional records.	Records attendance and creates assignments/assessments.	Checks attendance and completes assignments.
Learning support	Monitors academic activity.	Uploads materials and publishes notices.	Accesses resources and announcements.
Evaluation	Monitors institution-level metrics.	Evaluates submissions and enters marks.	Views marks, grades and results.

Stage	Admin	Faculty	Student
Examination	Manages institutional examination information.	Creates/publishes eligible online exams and schedules.	Attempts eligible exams and checks released results.

7.3 Attendance Workflow

1. Faculty selects the subject, date, time slot, semester and section.
2. The system loads or identifies the relevant class context and student records.
3. Faculty marks each student as present, absent or late and submits the session.
4. The backend validates the required fields and stores the attendance session.
5. Attendance percentages are recalculated for affected students.
6. Socket.IO events notify relevant clients that attendance information has changed.

7.4 Assignment Workflow

1. Faculty creates an assignment with title, subject, instructions, due date and maximum marks.
2. The assignment becomes available to the intended student cohort.
3. A student submits the required file/data and optional notes before or after the deadline, as supported by status handling.
4. The submission is stored with student and assignment references.
5. Faculty reviews the submission, awards marks and provides feedback.
6. The student can view the evaluation information in the portal.

7.5 Online Examination Workflow

1. Faculty prepares an online exam with questions, options, marks, duration, eligibility and publication settings.
2. The examination is published for the intended department, semester and section.
3. An eligible student starts an attempt; the system records attempt status and start time.
4. The student answers questions and may mark questions for review.
5. The attempt is submitted or reaches its expiry condition.
6. The system evaluates objective answers and calculates score-related information; result visibility follows the configured release mode.

The presentation specifies timed MCQ examinations and automatic evaluation. It also explicitly indicates that camera or webcam monitoring is not part of the current implementation.

8. FUNCTIONAL MODULES

8.1 Administrator Module

The Admin Portal provides institution-level management functions. It is responsible for maintaining the academic structure and monitoring the overall system.

- Manage department details and section records.
- Create and manage subjects, semester information and course details.
- Maintain student and faculty profiles and their account information.
- Assign faculty to subjects, semesters and sections through teaching assignments.
- Manage or review class timetables and examination schedules.
- Monitor attendance, academic records, analytics and audit information.
- Publish institutional announcements for the intended audience.

8.2 Faculty Module

The Faculty Portal supports teaching-related activities and provides faculty members with access to their assigned academic responsibilities.

- View assigned subjects, departments, semesters and sections.
- View relevant student lists and class information.
- Record attendance sessions and student attendance statuses.
- Create assignments and review student submissions.
- Create internal assessments and enter assessment marks.
- Create and manage online examinations, questions and results.
- Upload study materials and publish academic announcements.
- Review student performance information and weak-concept indicators.

8.3 Student Module

The Student Portal provides an individual view of academic information and learning activities.

- View personal profile, roll/registration number, department, semester and section.
- View enrolled subjects and faculty-related course information.
- Check timetable and examination schedules.
- View subject-wise and overall attendance information.
- View assignments, due dates and submission status.
- Submit assignment work and view faculty marks or feedback.
- Access study materials, links, notes and presentations.
- View announcements, notifications, internal marks, grades and results.
- Attend eligible online examinations and view results when released.

8.4 Department and Section Management

Department records contain department codes and names, head-of-department details, descriptions and summary counts. Section records associate a section name and code with a department, capacity, enrollment count and active/inactive status. These records provide the organizational context for student grouping and teaching assignments.

8.5 Subject and Teaching Assignment Management

Subject information includes subject code, name, department, semester, credits, subject type and syllabus topics. Teaching assignments connect faculty members with subjects and sections for a particular academic year. These relationships help determine which classes and academic tasks are relevant to a faculty member.

8.6 Timetable and Examination Schedule Module

Timetable entries define the day of the week, start and end times, subject, department, semester, section, room and faculty member. Examination schedule records include exam type, date, time, room, instructions and publication status.

8.7 Attendance Management

Attendance is represented as a session associated with a subject, department, semester, section, date, time slot and faculty member. Each session contains student-level records with present, absent or late status. The backend recalculates attendance for affected students and emits attendance-related events.

8.8 Assignment and Submission Module

Assignments contain academic instructions, subject and faculty details, target semester and section, due date and maximum marks. Submission records associate a student with an assignment and capture submission time, file details, optional notes, status, awarded marks and faculty feedback.

8.9 Internal Assessment and Results Module

Internal assessments support types such as Quiz, Unit Test, Mid-Term CIA and Lab Viva. Assessment mark entries store the marks obtained by students and optional feedback. Grade records include internal and external marks, total marks, grade, grade point and pass/fail status. Assessment result records include semester, SGPA, CGPA, total credits and subject-wise grade records.

8.10 Online Examination Module

The online examination module supports examination descriptions and instructions, department and subject mapping, semester and section eligibility, scheduled date and time, duration, total and pass marks, negative marking options, maximum attempts and result publication settings. Question records support options, correct answers, marks and optional explanations. Exam attempts store selected answers, marked-for-review questions, status, timing and evaluation details.

8.11 Study Material and Announcement Module

Study materials are associated with subjects and faculty members and include title, description, type, URL or file-related information, upload date and download count. Announcements include title, message, target audience, optional department/semester/section filters, publisher, publication or scheduling details and priority.

8.12 Analytics and Audit Module

Institutional analytics include counts of students, faculty, departments and courses, attendance-session totals, average attendance, students below the attendance threshold and students with low CGPA. Department metrics summarize student/faculty counts and average attendance. Weak-concept analytics aggregate assessment performance by topic tag or assessment title and identify concepts that may need intervention. Audit-log records capture actions, actor, role, target entity, details and timestamp.

9. DATABASE DESIGN

The backend defines Mongoose schemas for the principal academic entities. The persistent storage interface also maintains corresponding arrays in the JSON data store. The project therefore uses a logical document-oriented data model, with Mongoose schemas available for MongoDB synchronization.

9.1 Main Data Entities

Entity / collection concept	Purpose and representative information
User	Name, email, password hash, role, contact details, account status and creation date.
Department	Department code/name, HOD details, description and academic summary counts.
Section	Section name/code, department relationship, capacity, enrollment count and status.
Subject	Subject code/name, department, semester, credits, type, syllabus and assigned faculty details.
Teaching Assignment	Faculty, subject, department, semester, section, academic year and assignment status.
Student Profile	User relationship, roll/registration number, department, semester, section, batch, CGPA/SGPA and attendance percentage.
Faculty Profile	User relationship, employee ID, designation, department, qualification, specialization and assigned classes.
Course Enrollment	Student and subject relationship, credits, attendance totals and percentage, semester and academic year.
Timetable Entry	Day, start/end time, subject, department, semester, section, room and faculty.

Entity / collection concept	Purpose and representative information
Attendance Session	Class/date/time details and an array of student attendance records.
Assignment	Assignment title, subject, faculty, target cohort, instructions, due date and maximum marks.
Submission	Assignment/student relationship, submission time, file details, status, marks and feedback.
Internal Assessment	Assessment type, subject, faculty, cohort, date, maximum marks and topic tag.
Assessment Mark Entry	Assessment/student relationship, marks obtained, maximum marks and feedback.
Study Material	Subject/faculty association, title, description, type, URL/file information and upload date.
Exam Schedule	Exam type, subject, cohort, date/time, room, instructions and publication status.
Announcement	Title, message, audience, optional cohort filters, publisher, schedule and priority.
Audit Log	Action, actor, role, target entity, details and timestamp.
Online Exam	Exam configuration, eligibility, schedule, duration, marks, questions, attempts and result release settings.
Exam Attempt	Student/exam relationship, attempt number, answers, timing, status and evaluation output.

9.2 Logical Relationships

- A user account is associated with a role and may have a corresponding student or faculty profile.
- A department can contain multiple sections, subjects, student profiles and faculty profiles.
- A subject belongs to a department and is associated with a semester and credit value.
- A teaching assignment connects a faculty member to a subject and section.
- A student may have multiple course-enrollment records, attendance records, submissions, assessment marks and exam attempts.
- An attendance session belongs to a class context and contains multiple student attendance records.
- An assignment can have multiple student submissions.
- An internal assessment can have multiple student mark entries.
- An online examination can have multiple questions and multiple student attempts.
- Announcements and study materials can be targeted or associated with academic audiences and subjects.

9.3 Data Integrity Considerations

The source code defines TypeScript interfaces and Mongoose schema constraints for several entities. For a production database design, unique identifiers, indexes, required references, deletion rules, validation of marks and dates, and transaction or synchronization behavior should be reviewed. The JSON store and MongoDB synchronization path should also be tested for duplicate updates and partial synchronization.

9.4 Storage Mechanism

The storage module reads and writes the file server/data/edutrack_db.json. When MongoDB is connected, save operations initiate background synchronization using Mongoose model upserts. The API health and database-status endpoints expose the active connection state and record counts. The JSON file should be backed up and protected because it contains academic and account-related data.

10. TECHNOLOGY STACK

Technology	Role in Edu-Track
React	Builds reusable, interactive frontend components and role-specific portals.

Technology	Role in Edu-Track
TypeScript	Provides static types for user roles and academic data structures.
Vite	Provides frontend development and production build tooling.
HTML / CSS	Defines page structure, layout, styling and responsive presentation.
Node.js	JavaScript runtime for the backend application.
Express.js	Defines REST API routes and middleware.
MongoDB	Document database supported by the backend persistence design.
Mongoose	Defines schemas and performs MongoDB model operations.
JSON file storage	Provides local persistent storage and fallback operation.
JWT / jsonwebtoken	Creates and verifies signed application session tokens.
bcryptjs	Hashes passwords for secure credential storage.
Firebase Authentication	Provides Google sign-in for the administrator authentication flow.
Firebase Admin SDK	Supports server-side Firebase ID token verification when configured.
Socket.IO	Enables real-time server-to-client event communication.
Recharts	Supports chart-based data visualization in the frontend.
Lucide React	Provides interface icons.
Zod	Available validation library dependency; actual use should be confirmed per route before claiming comprehensive schema validation.

The package manifest also includes the Google GenAI SDK. Its presence in the dependency list alone does not establish a user-facing AI feature in the delivered application; such a feature should only be documented as implemented if a working integration is confirmed in the relevant application flow.

11. IMPLEMENTATION DETAILS

11.1 Frontend Structure

Source path	Responsibility
src/App.tsx	Main application component; manages current user, view state, navigation, notifications and real-time subscriptions.
src/components/auth/	Landing page and login interface.
src/components/admin/ AdminPortal.tsx	Administrator dashboard and academic administration screens.
src/components/faculty/ FacultyPortal.tsx	Faculty academic workspace.
src/components/student/ StudentPortal.tsx	Student academic workspace.
src/components/layout/	Shared header and sidebar/navigation components.
src/services/apiService.ts	Client-side API service, local cache and academic operations.

Source path	Responsibility
src/services/socketService.ts	Socket.IO client connection, registration and event subscriptions.
src/types/index.ts	TypeScript interfaces for users and academic entities.
src/config/firebase.ts	Firebase client initialization and Google sign-in helper.

11.2 Backend Structure

Source path	Responsibility
server/server.ts	Creates the Express application, HTTP server, middleware, API and Socket.IO integration.
server/routes/	Route modules for authentication, departments, sections, subjects, students, faculty, timetable, attendance, assignments, assessments, materials, announcements, analytics and online exams.
server/middleware/auth.ts	JWT token creation/verification and authentication/role middleware.
server/models/index.ts	Mongoose schemas and model definitions for academic entities.
server/db/connection.ts	MongoDB connection setup and connection-status helpers.
server/store/storage.ts	JSON file persistence, seed data, attendance recalculation, audit logging and MongoDB synchronization.
server/socket.ts	Server-side Socket.IO initialization and event emission helpers.
server/firebaseAdmin.ts	Server-side Firebase token verification support.

11.3 API Organization

The Express router groups endpoints by academic function. The exact endpoint list can be extended as the application evolves; the following table summarizes the route prefixes mounted in the supplied source.

API prefix	Functionality
/api/auth	Login and authentication-related operations.
/api/departments	Department records.
/api/sections	Section records.
/api/subjects	Subject records.
/api/students	Student records and profiles.
/api/faculty	Faculty records and profiles.
/api/timetable	Class timetable information.
/api/attendance	Attendance sessions and records.
/api/assignments	Assignment creation and submissions.
/api/assessments	Internal assessments and marks.
/api/materials	Study materials.
/api/announcements	Academic announcements.
/api/analytics	Institutional metrics, weak concepts and audit logs.

API prefix	Functionality
/api/online-exams	Online examination and attempt workflows.
/api/health and /api/database	Service health and persistence status endpoints.

11.4 Client-Side State and Synchronization

The frontend API service defines localStorage keys for user and academic data caches. The application also registers the signed-in user context with the Socket.IO client. When supported events arrive, the application updates cached information and displays notifications such as attendance updated, marks published, assignment created or material uploaded. The browser cache improves continuity and responsiveness, but should not be treated as a substitute for server-side authorization or verified persistent storage.

11.5 Local Setup (General)

1. Install a compatible Node.js runtime and npm.
2. Extract the project archive and open the project root in a terminal.
3. Install dependencies using npm install.
4. Review the environment example file and configure required values such as database connection and authentication settings. Do not publish secrets.
5. Start the application using the npm scripts defined in package.json (the root development script uses tsx server.ts).
6. Open the local URL printed by the development server in a modern browser.
7. To use MongoDB persistence, configure a reachable MongoDB URI and verify the connection using the database status endpoint.

The exact setup may vary by deployment environment. The supplied README is a brief AI Studio starter guide, so environment variables, production hosting and institutional configuration should be confirmed from the actual deployment setup before final submission.

12. AUTHENTICATION, SECURITY AND PRIVACY

12.1 Authentication

The backend authentication middleware supports JWT tokens. Tokens contain the user identifier, email, role and name, and the configured token lifetime in the source is seven days. The middleware reads a bearer token from the Authorization header or a token cookie. Protected routes can require authentication and restrict access to specified roles.

The frontend Firebase configuration integrates Google sign-in for the administrator flow. The server contains Firebase Admin support for verifying Firebase ID tokens when the service account configuration is available. The application also includes bcryptjs for password hashing.

12.2 Authorization

- Role-based authorization distinguishes admin, faculty and student users.
- Protected operations should be restricted at the backend, not only hidden in the frontend.
- Academic data access should be checked against department, semester, section, subject and user relationships.

12.3 Data Privacy

- Student academic records, attendance, marks and profile information should only be available to authorized users.

- Passwords must be stored as hashes and never returned through API responses.
- JWT secrets, Firebase service-account credentials and database credentials must be stored in environment configuration, not committed to source control.
- Audit logs can support accountability by recording who performed important operations.

12.4 Security Review Notes

The source code includes a fallback JWT secret in the authentication middleware. A production deployment must replace this with a strong secret supplied through protected environment configuration and should fail safely if the secret is missing. The Firebase client configuration contains public client configuration values; private service-account credentials must remain server-side. The archive includes a .env file, so it should be reviewed for secrets before sharing or publishing.

The presence of authentication middleware does not by itself prove that every route enforces the correct role and data ownership checks. Before production use, test unauthorized access, student-to-student data access, faculty access to unassigned classes, and cross-section or cross-department requests. Review CORS, token storage, rate limiting, input validation, file upload handling and audit coverage.

12.5 Backup and Recovery

The JSON persistence file and MongoDB database should be included in a documented backup process. Recovery testing should verify that academic records can be restored and that JSON-to-MongoDB synchronization does not create inconsistent or duplicated data.

13. TESTING STRATEGY AND TEST CASES

Testing should confirm that each user role can complete its intended tasks and cannot perform operations outside its permissions. The following cases are proposed validation scenarios based on the supplied modules; they are not claimed as executed test results.

Test ID	Scenario	Expected result
TC-01	Admin submits valid login credentials.	Admin session is established and Admin Portal is displayed.
TC-02	User submits invalid credentials.	Authentication is rejected with an appropriate error.
TC-03	Admin creates a department and section.	Valid records are stored and become available in relevant screens.
TC-04	Admin creates a subject and teaching assignment.	Subject and faculty/section association are saved correctly.
TC-05	Faculty views assigned subjects.	Only relevant teaching assignments are presented.
TC-06	Faculty records attendance for a valid class.	Attendance session is stored and attendance totals are recalculated.
TC-07	Faculty submits malformed attendance data.	The API rejects missing or invalid required fields.
TC-08	Faculty creates an assignment.	Assignment appears for the intended semester and section.
TC-09	Student submits assignment work.	Submission is associated with the correct student and assignment.
TC-10	Faculty grades a submission.	Marks and feedback are stored and visible to the appropriate student.
TC-11	Faculty publishes study material.	Material is available to its intended audience.
TC-12	Admin/faculty publishes an announcement.	Announcement is shown to the configured audience.

Test ID	Scenario	Expected result
TC-13	Student attempts an eligible online exam.	Attempt is created and timed examination flow is available.
TC-14	Student attempts an unpublished or ineligible exam.	The operation is rejected or exam is not made available.
TC-15	Online exam is submitted.	Attempt status and objective evaluation results are recorded.
TC-16	User accesses a protected route without a token.	The backend rejects the request where authentication is required.
TC-17	Student attempts an admin-only operation.	The backend denies access based on role.
TC-18	MongoDB is unavailable.	The application reports fallback storage mode and persists through the JSON store as designed.
TC-19	MongoDB becomes available.	Database connection status is updated and synchronization behavior is verified.
TC-20	Attendance or marks are updated while clients are connected.	Relevant Socket.IO events are received and the UI reflects the update.

13.1 Testing Types

- Unit testing: test individual calculations, validators, authentication helpers and academic business functions.
- Integration testing: verify frontend/API, route/storage, MongoDB synchronization and Socket.IO interactions.
- System testing: execute complete Admin, Faculty and Student workflows.
- Security testing: verify role checks, token validation, data ownership, input handling and secret management.
- Usability testing: confirm that common academic tasks can be completed with clear navigation.
- Recovery testing: verify backup restoration and behavior during database disconnection or synchronization failure.

13.2 Acceptance Criteria

The system can be considered ready for a controlled demonstration when the three roles can sign in, role-specific screens load, core academic records can be created and retrieved, attendance and assessment workflows complete, student-facing data is correctly scoped, online exam attempts follow configured rules, and the selected persistence mode is verified.

14. ADVANTAGES AND APPLICATIONS

14.1 Advantages

- Centralized academic information: key records are managed through one application.
- Role-based workspace: Admin, Faculty and Student users receive relevant functions.
- Reduced paperwork: attendance, assignments, marks and announcements can be handled digitally.
- Improved access: students can view academic details and resources through the portal.
- Academic monitoring: attendance and marks support review of student progress.
- Structured assessment: assignment submissions, internal marks and online examinations are organized.
- Timely communication: announcements and real-time events help distribute academic updates.
- Performance insights: weak-concept analytics can highlight areas for additional teaching support.
- Extensible architecture: modular frontend, backend routes and data models support future additions.

14.2 Applications

- Colleges and universities managing departments, semesters, sections and course activities.
- Engineering institutions managing theory subjects, laboratories, assignments and internal assessments.
- Academic departments monitoring attendance and student performance.
- Faculty teams distributing study resources and communicating course announcements.
- Institutions conducting objective online assessments and maintaining examination records.

15. LIMITATIONS AND FUTURE ENHANCEMENTS

15.1 Limitations and Considerations

- The current implementation uses a JSON-backed persistent store with optional MongoDB synchronization; production deployments should establish one authoritative persistence strategy and verify synchronization consistency.
- Actual authentication security depends on deployment configuration, secret management, Firebase verification settings and route-level authorization.
- Academic analytics depend on the completeness and correctness of attendance and assessment records.
- Online examination features include timed attempts and objective evaluation, but the presentation explicitly excludes camera/webcam proctoring.
- Study material file handling and storage capacity should be reviewed for production-scale uploads.
- Real-time updates depend on an active Socket.IO connection and appropriate event registration.
- The supplied README contains only basic setup guidance; deployment and operations documentation should be expanded.

15.2 Future Enhancements

- Introduce a fully standardized MongoDB-first persistence architecture with transactions, indexes, validation and reliable synchronization monitoring.
- Add automated timetable generation with faculty, room and class-conflict detection.
- Provide downloadable attendance, marks, results and analytics reports in PDF and spreadsheet formats.
- Add parent/guardian access with controlled visibility of attendance and academic progress.
- Introduce email, SMS or push notifications for urgent announcements, attendance shortages and examination reminders.
- Improve analytics with historical trends, cohort comparisons and configurable academic intervention thresholds.
- Add secure cloud file storage, file type validation, size limits and malware scanning for study materials and submissions.
- Expand automated testing and add continuous integration for build, lint, API and security checks.
- Provide a progressive web app or mobile application for easier student and faculty access.
- Add accessibility improvements, multilingual support and configurable institutional branding.
- Introduce examination question banks, randomized question selection and additional assessment formats.
- Add stronger audit reporting, backup automation and administrative data export.

16. CONCLUSION

Edu-Track – Student Academic Management System is designed to centralize academic administration and provide a common digital workspace for administrators, faculty members and students. Its modules cover institutional setup, student and faculty information, subjects, teaching assignments, timetables, attendance, assignments, internal assessments, online examinations, study materials, announcements and analytics.

The project demonstrates a modular full-stack implementation using React, TypeScript, Node.js and Express. The backend defines Mongoose schemas for MongoDB and includes a JSON-backed persistence layer with optional database synchronization. JWT-based authentication, Firebase Google sign-in integration and Socket.IO event communication support the system's identity and real-time interaction requirements.

By organizing academic records and workflows in one application, Edu-Track aims to reduce manual effort, improve information access and support more consistent academic monitoring. Further work on persistence consistency, production security, automated testing, reporting and deployment operations can strengthen the system for wider institutional use.

17. REFERENCES

1. Edu-Track project source code supplied for this report: project archive “edu-track – student academic management system”.
2. Edu-Track project presentation supplied by the project team: “EDU TRACK – Student Academic Management System”, 9 slides.
3. React documentation: https://react.dev/
4. TypeScript documentation: https://www.typescriptlang.org/docs/
5. Vite documentation: https://vite.dev/guide/
6. Node.js documentation: https://nodejs.org/docs/
7. Express documentation: https://expressjs.com/
8. MongoDB documentation: https://www.mongodb.com/docs/
9. Mongoose documentation: https://mongoosejs.com/docs/
10. JSON Web Token introduction: https://jwt.io/introduction
11. Firebase Authentication documentation: https://firebase.google.com/docs/auth
12. Socket.IO documentation: https://socket.io/docs/v4/

Note: This report follows the chapter organization and academic-report style of the supplied ResolveIT reference document. Project-specific descriptions were prepared from the Edu-Track presentation and source archive. Any implementation claim that depends on external configuration or live deployment should be validated in the final project environment.